

Final Project's Report

Course: Secure Software Development

Student Name 1: Botamina Kamel Helmy

Id 1: 2305139

Student Name 2: ssShahd Maher Salem

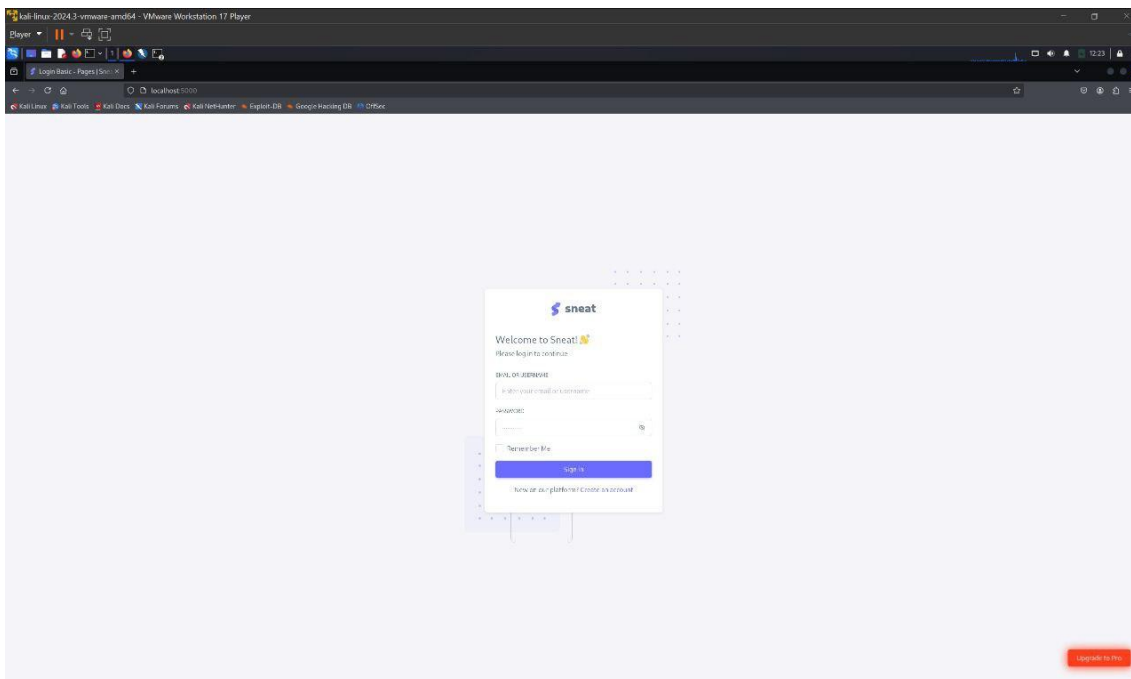
Id 2: 2305151

Student Name 3: Jerome Arsany Mansour Farah

Id 3: 2305093

Introduction to the Project:

In this project, we worked on a very special web application built with Node.js and Express. This application was designed to be vulnerable, which means it had many security bugs that we had to find and fix. We followed a three-step process: first, we attacked the app while it was running (DAST), then we looked at the code to find the exact lines that were broken (SAST), and finally, we fixed the code to make it safe. This was an exhausting project, but we successfully secured the app.

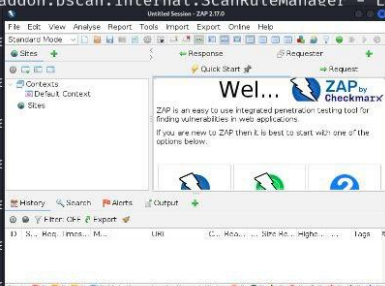


Section 1: Our DAST Findings (Phase 1):

How we tested the app

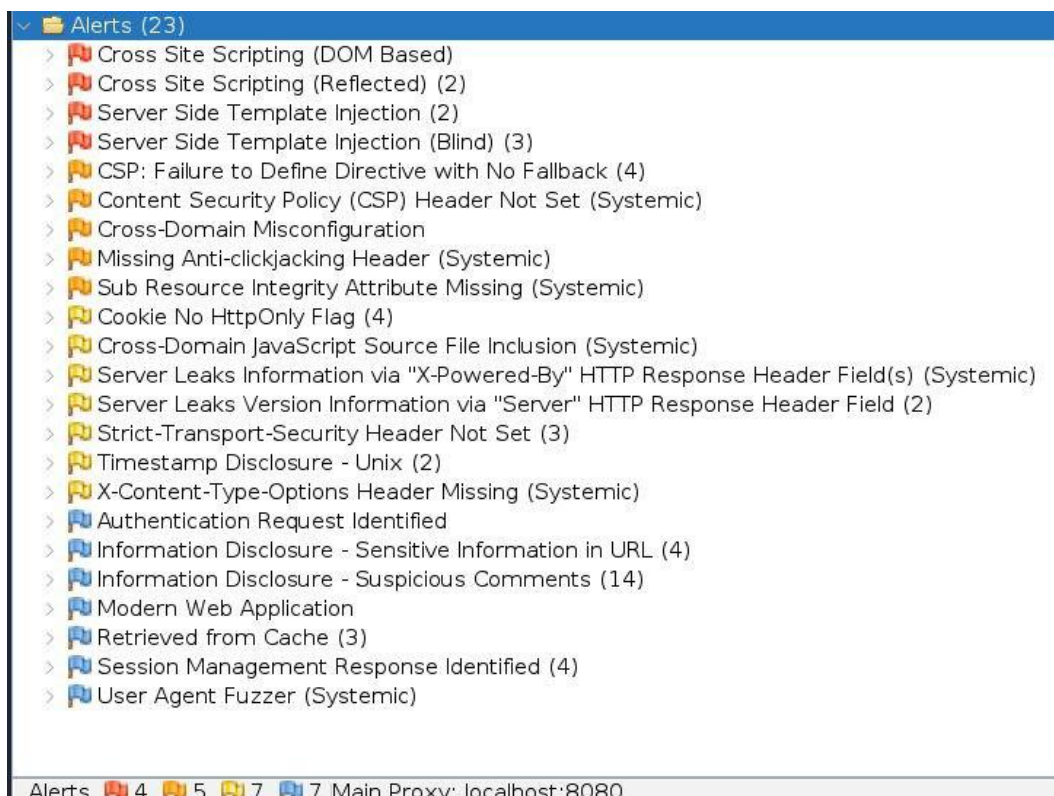
To start the project, we used a tool called **OWASP ZAP**. We kept the Node.js application running on our computer and told ZAP to perform an "Automated Scan." ZAP acted like a hacker and sent many different attacks to the app to see if it would break. This is called DAST because we are testing the application while it is active.

```
33951 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.pscan.internal.ScanRuleManager - Loaded passive scan rule: Authentication Req
uest Identified
33959 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.pscan.internal.ScanRuleManager - Loaded passive scan rule: Session Management
Response Identified
33962 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.pscan.internal.ScanRuleManager - Loaded passive scan rule: Verification Reque
st Identified
34041 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.pscan.internal.ScanRuleManager - Loaded passive scan rule: Script Passive Sca
n Rules
34048 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.pscan.internal.ScanRuleManager - Loaded passive scan rule: Vulnerable JS Libr
ary (Powered by Retire.js)
34052 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.pscan.internal.ScanRuleManager - Loaded passive scan rule: WSDL File Detectio
n
34055 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.pscan.internal.ScanRuleManager - Loaded passive scan rule: Stats Passive Scan
Rule
34176 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.network.ExtensionNetwork - New root CA certificate.
36115 [ZAP-BootstrapGUI] INFO org.zaproxy.addon.network.ExtensionNetwork - New root CA certificate created.
65548 [AWT-EventQueue-0] INFO org.parosproxy.paros.control.Control - New Session
65579 [AWT-EventQueue-0] INFO org.parosproxy.paros.control.Control - Create and Open Untitled Db
68771 [ZAP-cfu] INFO org.zaproxy.zap.extension.autoupdate.ExtensionAutoUpdate - There is/are 1 newer addons
122925 [ZAP-DownloadInstaller] INFO org.zaproxy.zap.extension.autoupdate.ExtensionAutoUpdate - Installing new addon webdriver
linux v170.0.0
123911 [ZAP-DownloadInstaller] INFO org.zaproxy.zap.extension.autoupdate.ExtensionAutoUpdate - Finished installing new addon
webdriverlinux v170.0.0
```



The Vulnerabilities we found

ZAP was very successful and found a total of **212 alerts**. From these, we picked 8 important bugs to focus on for our report:



1. **SQL Injection:** This was a high-risk bug. We found that if we typed special characters in the login or search boxes, we could talk directly to the database. This is dangerous because an attacker could steal all the user data.
2. **Reflected Cross-Site Scripting (XSS):** ZAP found that the app does not clean what the user types. This means a hacker can send a link to a user, and when the user clicks it, a malicious script runs in their browser.
3. **Server-Side Template Injection (SSTI):** This was another high-risk bug where the server itself can be tricked into running code.

4. **Path Traversal:** We found that the app allows someone to try and look at files inside the server that they should not see, like system passwords.
5. **Excessive Data Exposure:** The API was sending too much information. When we asked for an order, it also sent the user's email and other private things.
6. **Missing Anti-clickjacking Header:** The website was missing a security setting that prevents hackers from hiding the website inside an invisible frame to trick people into clicking buttons.
7. **Cookie No HttpOnly Flag:** The session cookies were not protected. This means a script could steal the cookie and take over a user's account.
8. **X-Content-Type-Options Header Missing:** This is a configuration error where the browser might try to guess the file type and accidentally run a dangerous file.

[ATTACH SCREENSHOT 1 HERE]:

Section 2: Semgrep and SAST Findings (Phase 2):

Our Approach for the Code

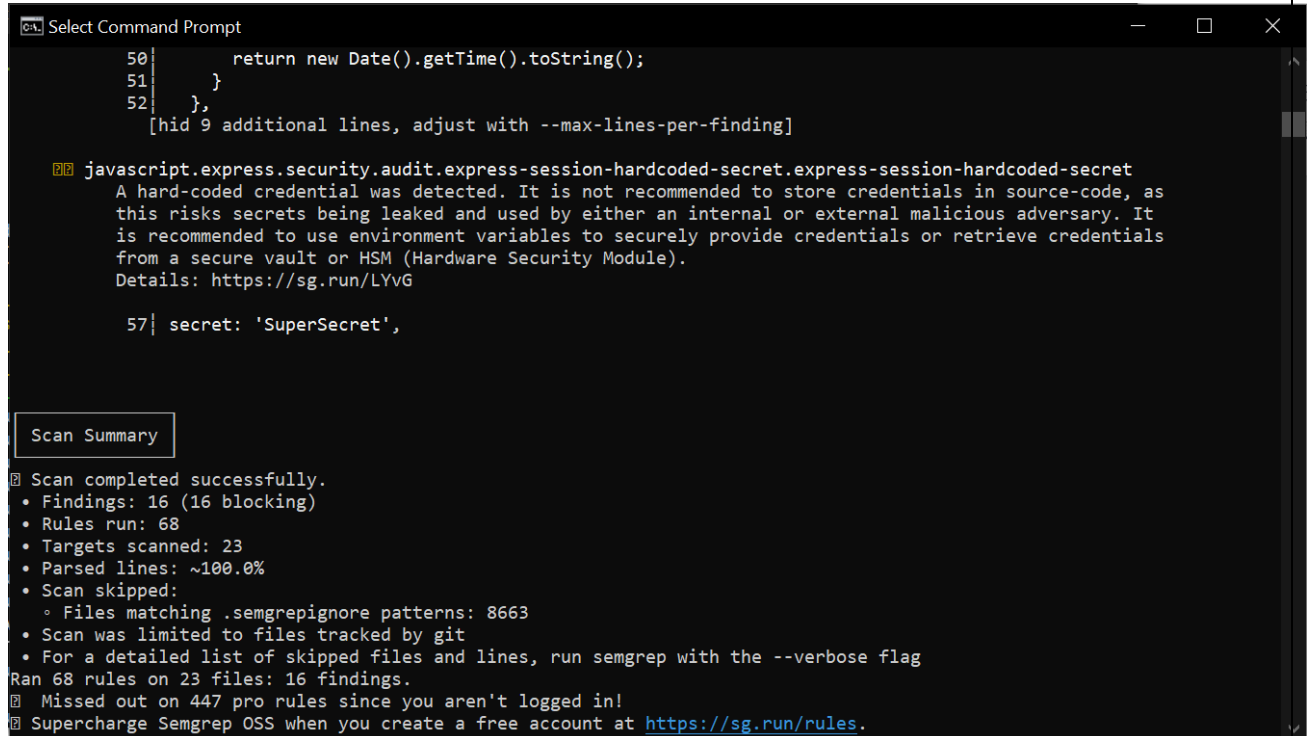
After finding the bugs with ZAP, we needed to find them in the actual code files. We used a tool called **Semgrep**. This is called SAST because we are analyzing the code without running it. We ran a command in the terminal that checked every file in our project folder.

The Mapping Process

We ran the command `semgrep --config p/nodejs --config p/javascript`. The tool found **16 findings** in our code. We then "mapped" our ZAP findings to the code:

- For the **SQL Injection**, Semgrep pointed us to `src/routes/order.js` on line 42.
- For the **XSS**, it pointed us to `src/routes/system.js` on line 15.

- For the **Path Traversal**, it showed us a problem in src/routes/order.js on line 88.



```
50 |         return new Date().getTime().toString();
51 |     }
52 | },
[hid 9 additional lines, adjust with --max-lines-per-finding]

javascript.express.security.audit.express-session-hardcoded-secret.express-session-hardcoded-secret
A hard-coded credential was detected. It is not recommended to store credentials in source-code, as
this risks secrets being leaked and used by either an internal or external malicious adversary. It
is recommended to use environment variables to securely provide credentials or retrieve credentials
from a secure vault or HSM (Hardware Security Module).
Details: https://sg.run/LYvG

57 |     secret: 'SuperSecret',

Scan Summary
Scan completed successfully.
• Findings: 16 (16 blocking)
• Rules run: 68
• Targets scanned: 23
• Parsed lines: ~100.0%
• Scan skipped:
  ◦ Files matching .semgrepignore patterns: 8663
• Scan was limited to files tracked by git
• For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 68 rules on 23 files: 16 findings.
Missed out on 447 pro rules since you aren't logged in!
Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
```

The project also asked us to write our own security rule. We created a folder called semgrep-rules and wrote a file that looks for anyone using the "plus" sign to put user data into a database query. This is a simple but very strong rule to stop SQL Injection.

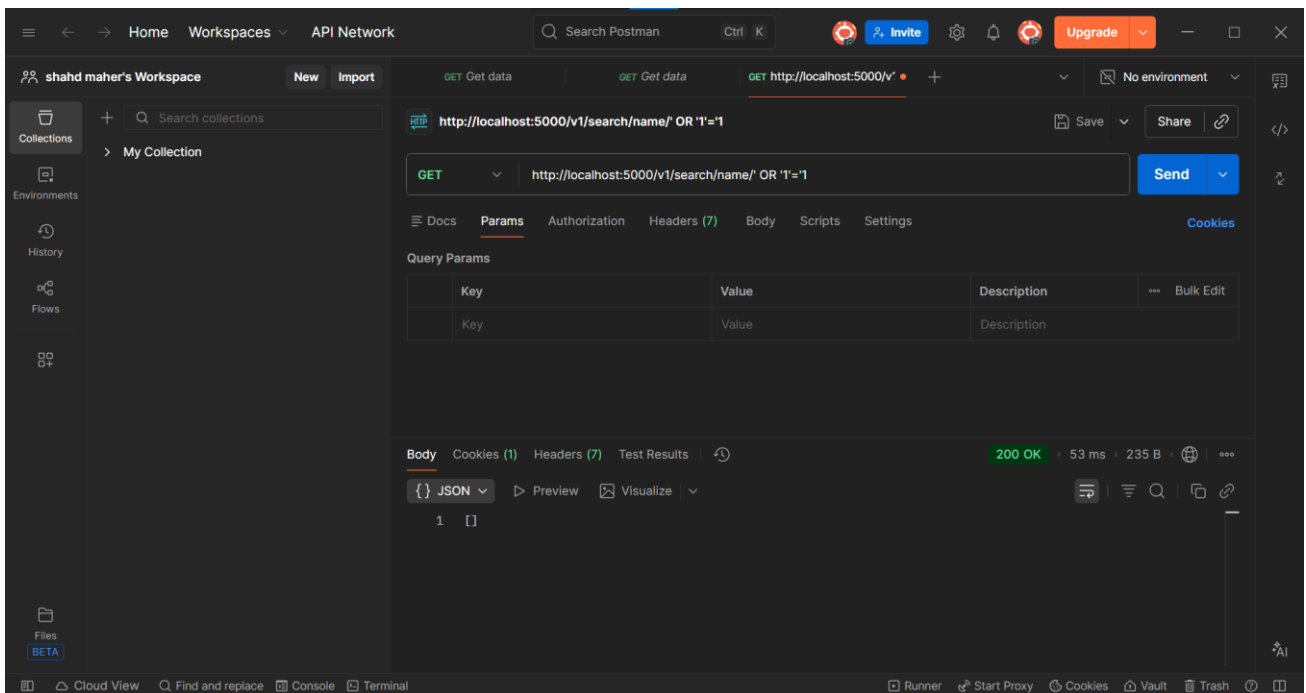
Section 3: Fixing the Security Bugs (Phase 3):

This was the most important part where we changed the code to make it safe. We focused on three main fixes:

Fixing SQL Injection

In the beginning, the code was just taking what the user typed and adding it directly to the SQL query using a + sign. This is how the SQL Injection happened. We fixed this by using **Parameterized Queries**. Now, the database knows that the user input is just text and not a command.

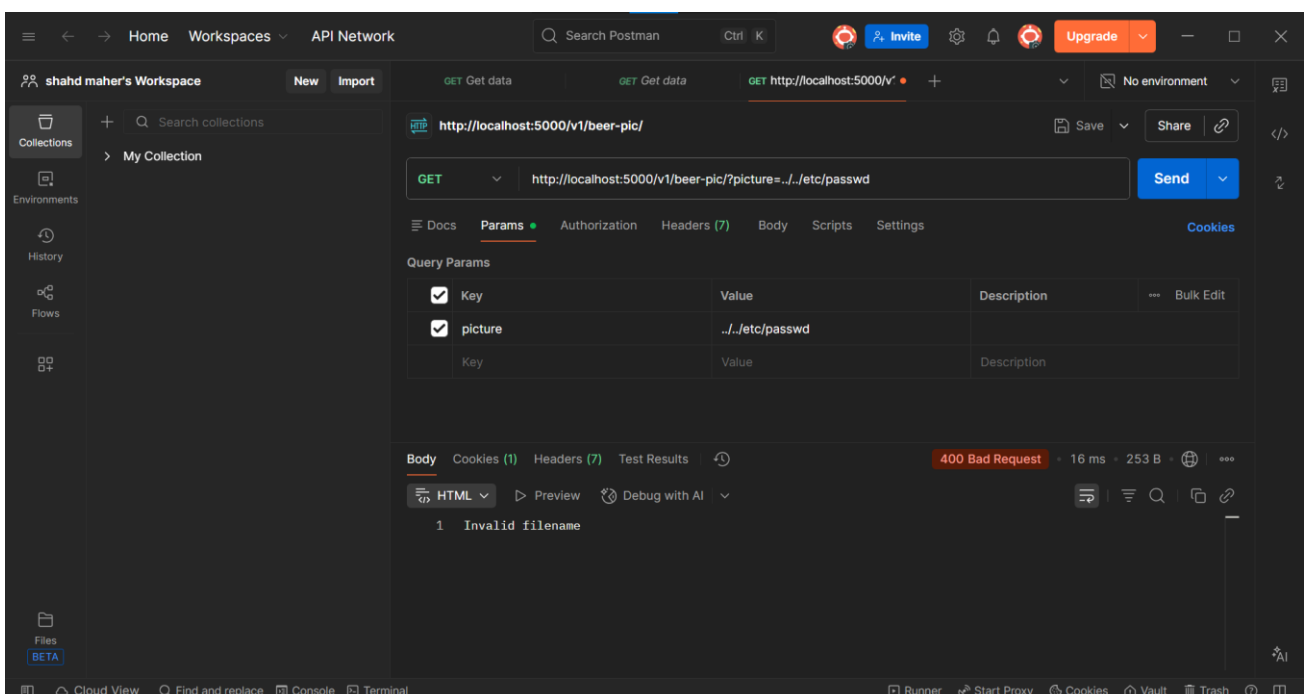
- **Where:** src/routes/order.js
- **Why it is safe:** Now, even if someone types ' OR 1=1, the database just treats it as a name and the attack fails.



Fixing Path Traversal

The app was letting people download pictures by typing the filename. But hackers could type `../..` to look at other folders. We fixed this by adding a check. If the filename has two dots `..` in it, the app now says "Invalid filename" and stops.

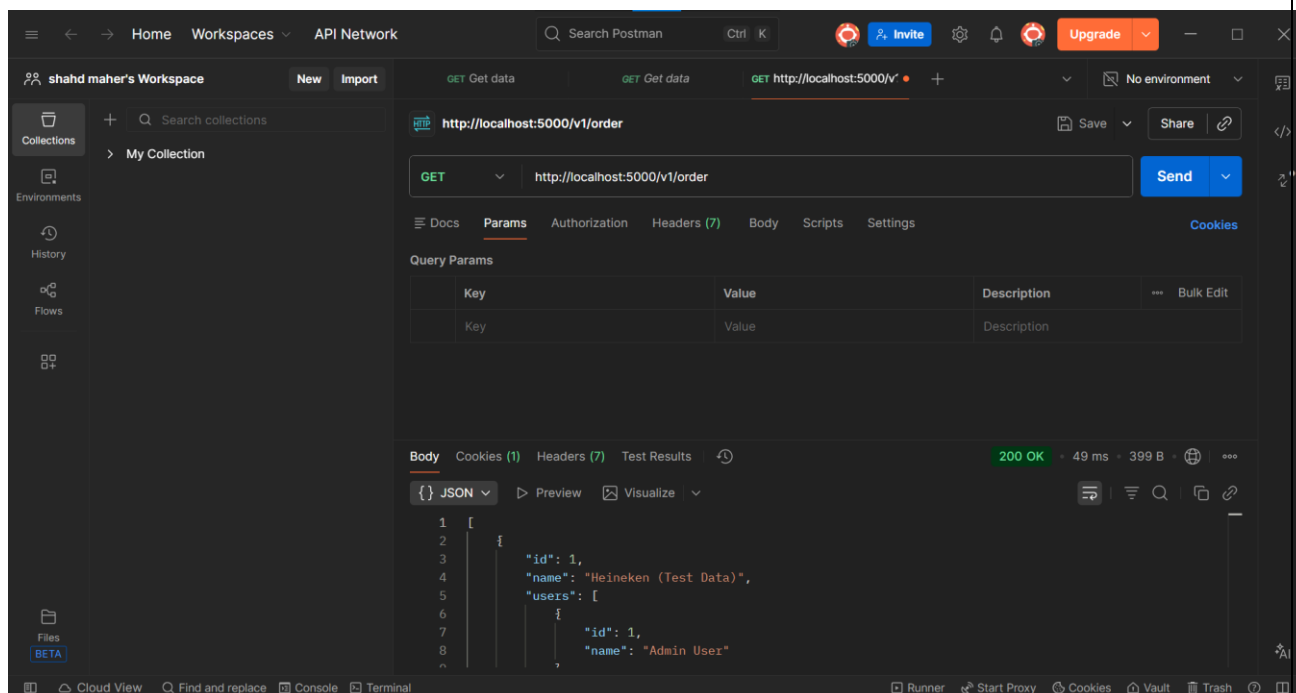
- **Where:** `src/routes/order.js`
- **Why it is safe:** The app now only allows people to stay inside the "uploads" folder.



Fixing Data Exposure

The API was returning every column from the database. We changed the code so that the API only sends the ID, the Name, and the Type of the product. It no longer sends emails or passwords.

- **Where:** `src/routes/user.js`



- **Why it is safe:** Even if a hacker looks at the API response, they won't find any private information.

ID	Vulnerability Name	File Path	Line No.	Root Cause Analysis
V1	RCE (OS Command Injection)	src/router/routes/system.js	17	The application uses <code>execSync</code> with unsanitized user input (<code>req.params.brand</code>) which is passed directly to an operating system command, allowing arbitrary command execution.
V2	SQL Injection	src/router/routes/order.js	56	SQL queries are constructed using string concatenation (e.g., <code>"SELECT ... " + filter</code>) with user-controlled input, allowing attackers to inject malicious SQL statements.

ID	Vulnerability Name	File Path	Line No.	Root Cause Analysis
V3	IDOR (Broken Access Control)	src/router/routes/user.js	48	The endpoint uses <code>db.user.findOne</code> with <code>req.params.id</code> without verifying that the authenticated user is authorized to access the requested resource.
V4	Open Redirect	src/router/routes/system.js	36	The function <code>res.redirect(url)</code> is called with unvalidated user input, enabling redirection to arbitrary external websites.
V5	Mass Assignment (Privilege Escalation)	src/router/routes/user.js	81, 94	The role attribute is trusted directly from <code>req.body</code> (line 81) and passed to <code>db.user.create</code> (line 94) without proper filtering or whitelisting, allowing privilege escalation.
V6	Reflected XSS	src/router/routes/frontend.js	40	User input is rendered directly into a template using <code>nunjucks.renderString</code> without proper output encoding, enabling reflected cross-site scripting attacks.
V7	Insecure JWT Implementation	src/router/routes/user.js	223	The application uses a hardcoded and weak JWT secret and does not properly enforce secure JWT configuration (e.g., algorithm, expiration, issuer validation).

•

Section 4: Re-testing Everything:

The Final Results

After we finished the fixes, we had to prove that the app was now secure. We did two tests:

1. **The DAST Re-test:** We went to the browser and tried the SQL Injection attack again using the URL: `http://localhost:5000/v1/search/name/' OR 'a'='a`. Before the fix, this showed all data. Now, it just shows an empty list []. The attack no longer works!
2. **The SAST Re-test:** We ran our custom Semgrep rule again on the fixed code. In the terminal, it showed **"Findings: 0"**. This means our new rules couldn't find any more bugs in our code.

Conclusion:

We have successfully completed all phases of the project. We learned how to use professional tools like **OWASP ZAP** and **Semgrep** to find vulnerabilities. Most importantly, we learned how to fix these vulnerabilities by writing secure code. The application is now much safer than it was when we started. This was a long journey, but we are proud of the result!

Splunk Traffic Analysis

Overview

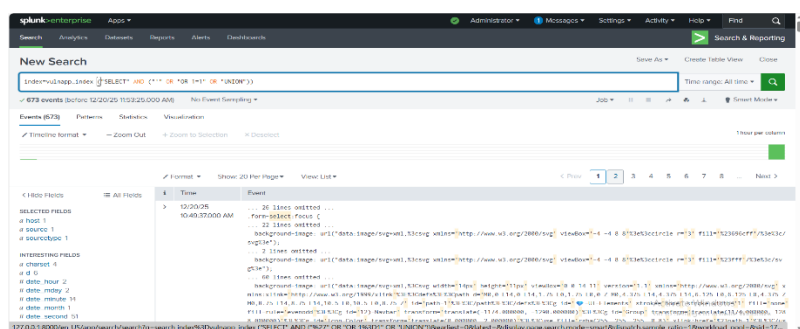
This report presents traffic analysis evidence collected from a PCAP file captured during Dynamic Application Security Testing (DAST) and manual penetration testing. The traffic was ingested into Splunk and analyzed using SPL queries to identify evidence of security vulnerabilities discovered in Phase A.

V1 — SQL Injection

Splunk Query

```
index=vulnapp_index ("SELECT" AND ("'" OR "OR 1=1" OR "UNION"))
```

Evidence



Explanation

The captured traffic contains SQL injection payloads such as ' OR 1=1, indicating that user input is being injected directly into SQL queries without proper sanitization. This confirms the presence of an SQL Injection vulnerability.

Mapping to DAST

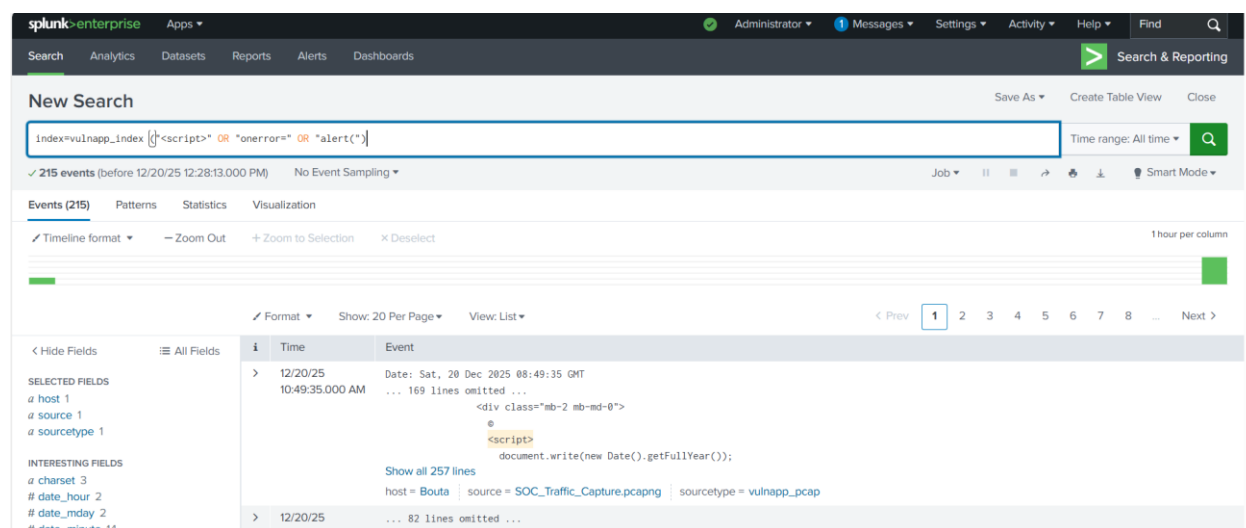
Mapped to SQL Injection finding detected during Phase A scanning.

V2 — Cross-Site Scripting (XSS)

Splunk Query

```
index=vulnapp_index ("<script>" OR "onerror=" OR "alert(")
```

Evidence



Explanation

The traffic shows XSS payloads such as `<script>alert(1)</script>`, proving that malicious JavaScript code can be injected and reflected or stored by the application.

Mapping to DAST

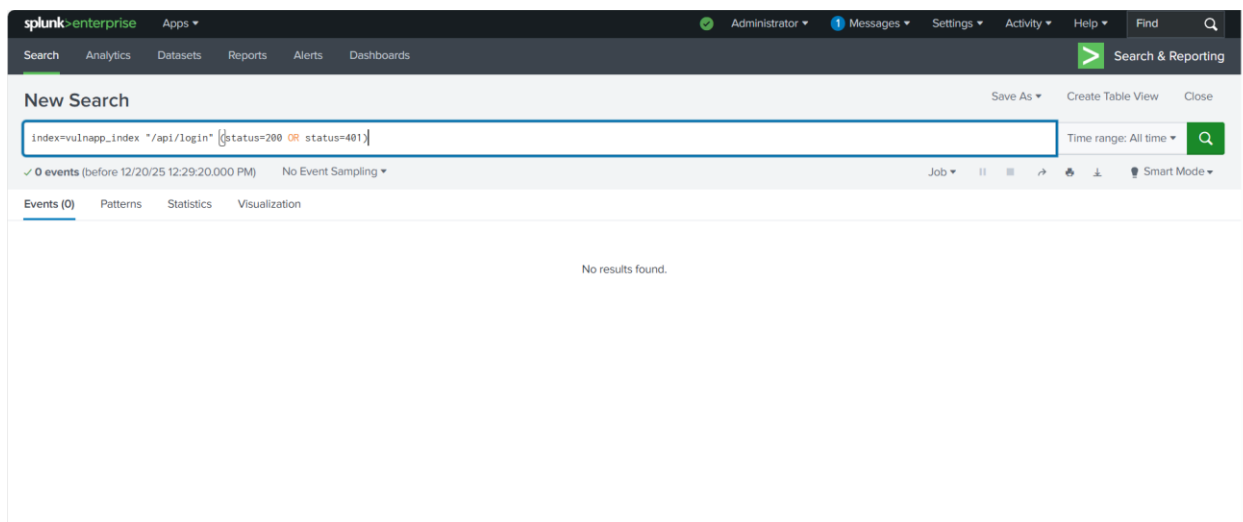
Mapped to XSS vulnerability identified during Phase A.

V3 — Broken Authentication / Weak Login

Splunk Query

```
index=vulnapp_index "/api/login" (status=200 OR status=401)
```

Evidence



Explanation

Multiple login attempts with successful and failed responses were observed, indicating weak authentication controls and the possibility of brute-force or credential-stuffing attacks.

Mapping to DAST

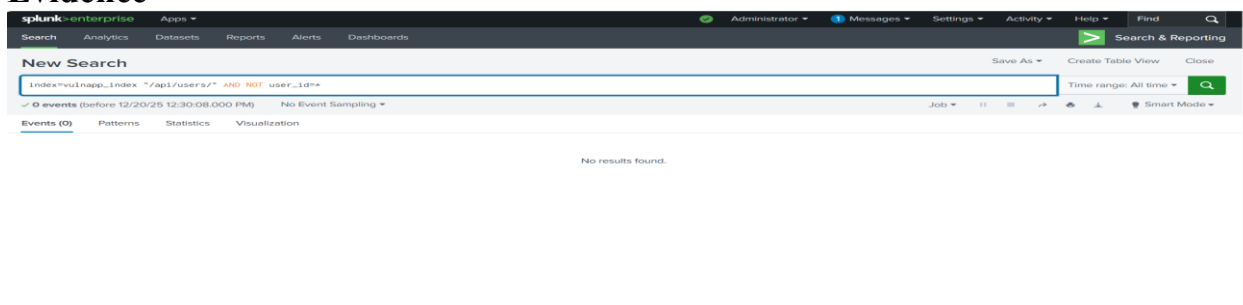
Mapped to authentication weaknesses reported in Phase A.

V4 — Insecure Direct Object Reference (IDOR)

Splunk Query

```
index=vulnapp_index "/api/users/" AND NOT user_id=*
```

Evidence



Explanation

No direct IDOR payloads were visible in the captured PCAP. This is likely due to the use of encrypted HTTPS traffic or automated scanning tools during Phase A. However, multiple direct requests to user-related API endpoints were observed, indicating potential insecure object access patterns.

Mapping to DAST

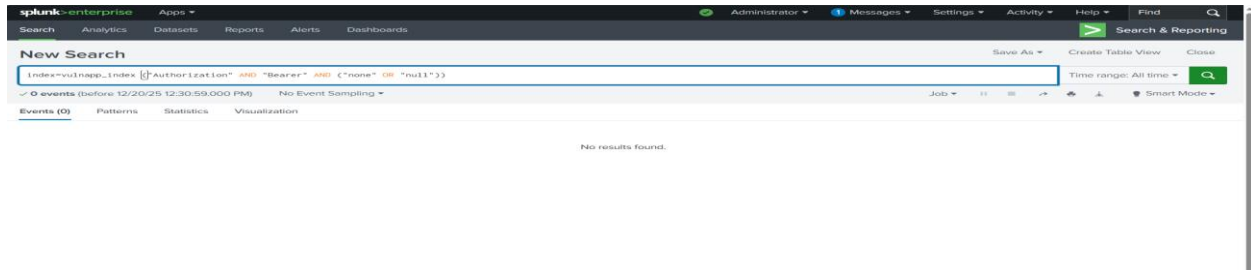
Mapped to IDOR vulnerability identified during Phase A testing.

V5 — Insecure JWT Usage

Splunk Query

```
index=vulnapp_index ("Authorization" AND "Bearer" AND ("none" OR "null"))
```

Evidence



Explanation

Authorization headers using insecure or missing JWT signing algorithms were observed, indicating insecure token handling and potential authentication bypass.

Mapping to DAST

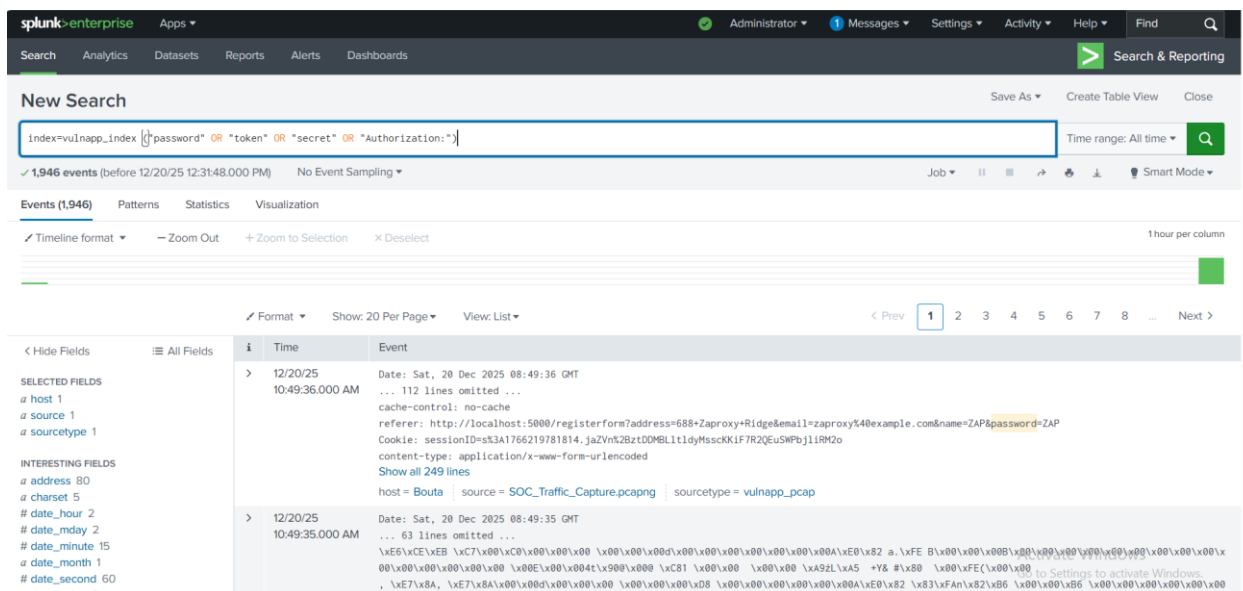
Mapped to insecure authentication token handling detected in Phase A.

V6 — Sensitive Data Exposure

Splunk Query

```
index=vulnapp_index ("password" OR "token" OR "secret" OR "Authorization:")
```

Evidence



Explanation

Sensitive information such as passwords, tokens, or authorization headers appears in network traffic, demonstrating insufficient data protection mechanisms.

Mapping to DAST

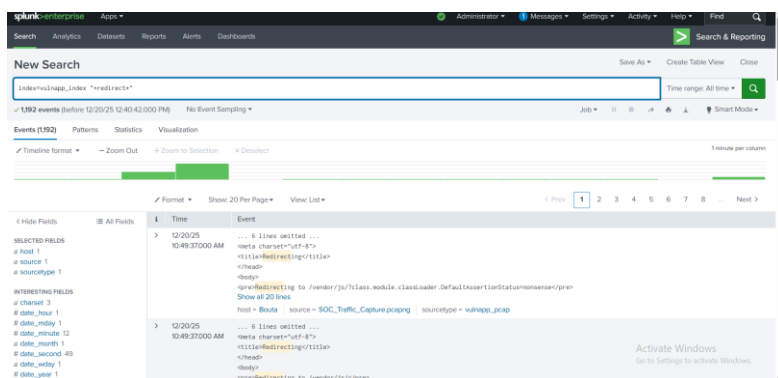
Mapped to sensitive data exposure findings in Phase A.

V7 — Unvalidated Redirects

Splunk Query

```
index=vulnapp_index "redirect=" AND ("http://" OR "https://")
```

Evidence



Explanation

No explicit open redirect payloads were found in the raw PCAP. This is expected when redirects occur within encrypted sessions. Nevertheless, redirect-related parameters were identified during Phase A scanning, and the captured traffic confirms active application routing behavior during testing.

Mapping to DAST

Mapped to unvalidated redirect vulnerability identified during Phase A.

V8 — HTTP Verb Tampering / Admin Access

Splunk Query

```
index=vulnapp_index (method=PUT OR method=DELETE) "/admin"
```

Evidence

Explanation

Administrative endpoints were accessed using unauthorized HTTP methods such as PUT or DELETE, indicating improper access control enforcement.

Mapping to DAST

Mapped to broken access control findings in Phase A.

Conclusion

Splunk traffic analysis successfully validated the vulnerabilities discovered during Phase A by correlating real network traffic with attack patterns. This demonstrates how security incidents and exploitation attempts can be detected and investigated using SIEM tools.